Knowledge Graph Design Document (KGDD)

IntelliPlant AI

AI-Powered Industrial Knowledge Intelligence Platform

Version: 1.0
Document Type: Knowledge Graph Design Document (KGDD)
Prepared For: ET AI Hackathon 2026
Prepared By: Team IntelliPlant AI

Table of Contents

---

1. Introduction

The Knowledge Graph serves as the central knowledge layer of IntelliPlant AI. It models industrial entities and their relationships, enabling explainable multi-hop reasoning and GraphRAG.

Technology Stack:

- Neo4j
- APOC Procedures
- LangChain Graph Transformer
- LangGraph

- Gemini/OpenAI

---

2. Objectives

The graph should:

- Connect fragmented documents.
- Preserve organizational knowledge.
- Support GraphRAG.
- Enable multi-hop reasoning.
- Improve explainability.
- Reduce hallucinations.

---

3. High-Level Graph Architecture

```text id="9v6a5r" Documents │ OCR │ Entity Extraction │ Relationship Extraction │ Graph Builder │ Neo4j │ GraphRAG │ LLM │ Response

```
---

4. Domain Ontology

The ontology models industrial assets, failures, people, and regulations.

Entity Categories

Asset Domain

Equipment

Component

Plant

Sensor

Process

Maintenance Domain

```
```

```
Incident

Failure

Inspection

WorkOrder

MaintenanceTask

---

### Human Domain

Operator

Engineer

Technician

Team

---

### Knowledge Domain

Manual

SOP

Procedure

Report

Document

---

### Compliance Domain

Regulation

Standard

Audit

CAPA
```

````
---

# 5. Node Types

## Equipment

Examples

Pump P101

Motor M201

Valve V301

Boiler B401

---

Properties

```json id="fh9weu"
{
 "equipment_id":"",
 "name":"",
 "type":"",
 "manufacturer":"",
 "model":"",
 "plant":"",
 "status":""
}
````

Failure

Examples

Bearing Failure

Leakage

Overheating

Corrosion

````

Properties

```json id="votd4e" { "failure_id":"", "severity":"", "category":"", "timestamp":"" }

```

Incident

Properties

```json id="ktv4sy"
{
 "incident_id":"",
 "description":"",
 "date":"",
 "location":""
}
```

Manual

Properties

```json id="pb0k8w" { "manual_id":"", "name":"", "version":"", "source":"" }

```

Inspection

Properties

```json id="zjzmpq"
{
 "inspection_id":"",
 "date":"",
 "inspector":"",
 "status":""
}
```

Operator

Properties

```json id="57vbd3" { "operator_id":"", "name":"", "department":"" }

````

Regulation

Properties

```json id="2zup1d"
{
 "regulation_id":"",
 "standard":"",
 "section":""
}
````

---

6. Relationship Types

CONNECTED_TO

Equipment → Equipment

Example

Pump → Motor

FAILED_IN

Equipment → Failure

Example

Pump P101 → Bearing Failure

CAUSES

Failure → Incident

Example

Bearing Failure → Overheating

DESCRIBED_BY

Equipment → Manual

INSPECTED_BY

Inspection → Operator

REFERENCES

Manual → Regulation

REQUIRES

Equipment → MaintenanceTask

SIMILAR_TO

Incident → Incident

LOCATED_IN

Equipment → Plant

REPORTED_BY

Incident → Engineer

---

7. Property Graph Schema

```text id="1jstn8" Equipment │ ├── equipment_id ├── type ├── manufacturer ├── model └── plant

Failure │ ├── severity ├── category └── timestamp

Manual │ ├── version ├── source └── file_path
```

````
---

# 8. Entity Extraction Pipeline

```text id="b7yq8w"
PDF
 ↓
OCR
 ↓
Chunking
 ↓
LLM Entity Extraction
 ↓
Structured JSON
 ↓
Graph Builder
 ↓
Neo4j
````

Example

Input

```text id="ztb0t2" Pump P101 experienced bearing failure on 12 March 2025.

````
Extracted

```json id="u7a1yy"
{
 "equipment":"Pump P101",
 "failure":"Bearing Failure",
 "date":"2025-03-12"
}
````

---

9. Relationship Extraction

Input

```text id="38zmpw" Bearing failure caused overheating.

````
Output

```text id="1jlwmh"
Pump P101

↓

FAILED_IN

↓

Bearing Failure

↓

CAUSES

↓

Overheating
````

---

10. Graph Construction Pipeline

```text id="3hkhxm" Document Upload ↓ OCR ↓ Entity Extraction ↓ Relationship Extraction ↓ Node Deduplication ↓ Cypher Generation ↓ Neo4j

````
---

11. Cypher Examples

Create Equipment

```cypher id="1w0v1s"
CREATE (:Equipment{
id:'P101',
name:'Pump P101'
})
````

Create Failure

```cypher id="9n1hmi" CREATE (:Failure{ name:'Bearing Failure' })

````

Create Relationship

```cypher id="fgv6l6"
MATCH (e:Equipment{name:'Pump P101'}),
(f:Failure{name:'Bearing Failure'})
CREATE (e)-[:FAILED_IN]->(f)
````

---

12. Graph Traversal

Query

Why did Pump P101 fail?

Traversal

```text id="9n3ee0" Pump P101

↓

FAILED_IN

↓

Bearing Failure

↓

CAUSES

↓

Overheating

↓

SIMILAR_TO

↓

Incident 2024-08

```
---

13. Multi-Hop Reasoning

Traditional Search

```text id="c3j2fe"
Pump

↓

Chunk

↓

Answer
```

Graph Search

```text id="ayxij3" Pump

↓

Failure

↓

Incident

↓

Manual

↓

Inspection

↓

Recommendation

```

```

```
# 14. GraphRAG Integration

```text id="cf0j0f"
User Query

↓

Entity Detection

↓

Neo4j Traversal

↓

Graph Expansion

↓

Vector Search

↓

Context Fusion

↓

Gemini

↓

Answer
```

---

15. Graph Embeddings

Future Support

Node2Vec

GraphSAGE

FastRP

Used For
```

Similarity

Recommendations

Failure prediction

---

16. Knowledge Graph Memory

Long-Term Memory

Neo4j

Stores

Historical incidents

Failures

Relationships

Short-Term Memory

Redis

Stores

Current session

Semantic Memory

Qdrant

Stores

Embeddings

---

17. Confidence Scoring

Graph Confidence

```text id="ymh8j5" Confidence = 0.5 Graph Evidence + 0.3 Retrieval Score + 0.2 Citation Score

```
---

18. Graph Analytics

Supported Metrics

Degree Centrality

Betweenness Centrality

PageRank

Community Detection

Shortest Path

Similarity

Example

Identify critical equipment:

```cypher id="g1xv6h"
CALL gds.pageRank.stream()
```

---

19. Graph Visualization

Frontend

React Flow

Neo4j Bloom

Cytoscape.js

Example

```text id="mrg2tk" Pump P101 | FAILED_IN | Bearing Failure | CAUSES | Overheating
```

````
---

# 20. Example Query Patterns

### Find Similar Failures

```cypher id="7x0mvg"
MATCH (f:Failure)<-[:FAILED_IN]-(e)
RETURN e,f
````

Equipment History

```cypher id="50rtz0" MATCH (e:Equipment)-[*]->(n) RETURN e,n

````

Compliance Lookup

```cypher id="ym6pd3"
MATCH (m:Manual)-[:REFERENCES]->(r:Regulation)
RETURN m,r
````

---

21. Scalability

Millions of nodes.

Horizontal Neo4j cluster.

Incremental graph updates.

Streaming ingestion.

APOC procedures.

Caching layer.

````

22. Future Enhancements

Industrial Ontology

Temporal Knowledge Graph

Graph Neural Networks

Self-Updating Graphs

Digital Twin Graph

Predictive Maintenance Graph

Supply Chain Graph

IoT Graph

Complete Graph Pipeline

```
text id="j7p0r7"
Documents
↓
OCR
↓
Entity Extraction
↓
Relationship Extraction
↓
Knowledge Graph Builder
↓
Neo4j
↓
GraphRAG
↓
Multi-Agent System
↓
LLM
↓
Response
```

Conclusion

The Knowledge Graph forms the foundation of IntelliPlant AI's Industrial Operations Brain. It captures relationships between equipment, failures, manuals, inspections, and regulations, enabling GraphRAG, explainability, and multi-hop reasoning that significantly improves answer quality and reduces hallucinations.